

PROJECT PROPOSAL

Level 02



ChainOps

Smart Contract DevSecOps and Operations Platform

IntelliCoders (04)

Faculty of Information Technology

University of Moratuwa

2026

PROJECT PROPOSAL

Level 02



ChainOps

Smart Contract DevSecOps and Operations Platform

IntelliCoders (04)

Name	Index Number
W.W.A.S.T Fernando	244063P
L.L.K.N Rathnasiri	244176N
J.M.D.S Jayasiri	244087R
J.B.A.I.N Jayawickrama	244092D
F.F Fazreen	244059J

Supervisor: Ms MGNN PREAMADASA

.....
Signature of the Supervisor

21/09/2026
.....
Date

Faculty of Information Technology

University of Moratuwa

2026

CONTENTS

1. Introduction	1
2. Background & Motivation	1
3. Problem in Brief - Fragmented Smart-Contract Lifecycle	1
4. Aim & Objectives	1
5. Proposed Solution - ChainOps	2
6. Resource Requirements	4
7. Work Distribution	5
8.Reference	5
9.Appendix	6

1. Introduction

Blockchain smart contracts are software programs deployed to blockchain networks and are commonly developed through a lifecycle that includes source-code management, compilation, automated testing, security analysis, deployment, and post-deployment monitoring. In practice, these activities may be performed using separate tools, command-line utilities, scripts, blockchain RPC services, and monitoring interfaces. This fragmentation makes it difficult for a development team to maintain one clear and traceable view of how a particular source-code revision was validated, approved, deployed, and monitored.

ChainOps is proposed as a developer-focused Smart Contract DevSecOps and operations platform for EVM-compatible smart-contract projects. It connects GitHub-based development with an automated and visible workflow for compilation, testing, security validation, policy evaluation, controlled deployment, contract registration, runtime monitoring, alerts, notifications, and audit history.

ChainOps does not create a new blockchain, Solidity compiler, security analyzer, or source-code hosting service. Instead, it integrates established tools and services and provides the orchestration, governance, traceability, and operational-control layer around them. GitHub remains the source of truth for project source code, while ChainOps manages lifecycle information associated with each relevant source-code revision.

2. Background & Motivation

A smart-contract development team may use GitHub for source control, Hardhat or Foundry for compilation and testing, Slither or another established analyzer for security analysis, deployment scripts and EVM RPC services for blockchain interaction, and separate tools for observing deployed contracts. Although each tool solves an important part of the process, the overall lifecycle can remain fragmented.

A team may need to determine which exact source-code commit was compiled and tested, what security findings existed for that revision, whether it satisfied release policies, who approved a deployment, which blockchain transaction performed the deployment, what contract address was produced, and what important runtime events occurred afterward. When these records are distributed across repositories, command-line outputs, scripts, wallets, blockchain explorers, and monitoring services, maintaining this traceability becomes difficult.

The motivation for ChainOps is therefore not to replace mature blockchain-development tools, but to connect them through a controlled lifecycle in which a smart-contract revision can be followed from a GitHub commit through validation and governance to deployment and runtime monitoring.

3. Problem in Brief - Fragmented Smart-Contract Lifecycle

Smart-contract teams often perform source management, compilation, testing, security analysis, deployment, and monitoring through separate tools and processes. This makes it difficult to maintain a single traceable relationship between a source-code revision, validation results, security status, approval decision, deployment transaction, deployed contract, and subsequent runtime activity. ChainOps addresses this problem by providing a centralized lifecycle-management and orchestration platform while continuing to use established tools for the specialized work they already perform.

4. Aim & Objectives

4.1 Aim

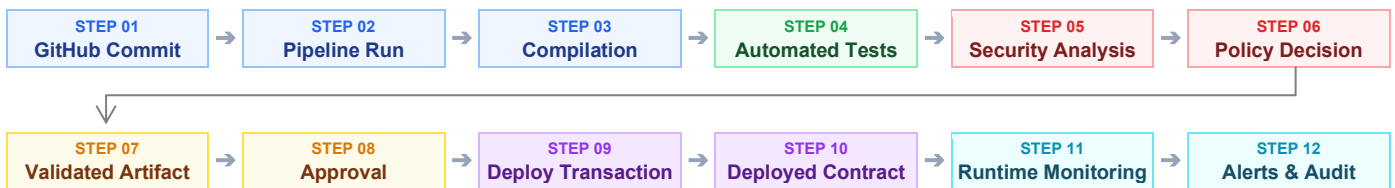
The aim of this project is to design and develop a Smart Contract DevSecOps and operations platform that connects GitHub-based smart-contract development with automated validation, security policy enforcement, controlled deployment, contract traceability, and post-deployment monitoring for EVM-compatible projects.

4.2 Objectives

- Study the smart-contract development, validation, deployment, and monitoring workflow and relevant technologies.
- Design and implement authentication, organization/project management, environments, and role-based access.
- Integrate GitHub repositories and repository events with a visible smart-contract pipeline workflow.
- Automate compilation, testing, and established security-analysis tools and collect their results.
- Apply policy-based release gates to validation and security results.
- Provide approval and governance workflows for sensitive deployment operations.
- Deploy validated smart-contract revisions to configured EVM development/test networks and track transaction status and confirmations.
- Maintain a versioned Contract Registry linking source commits, artifacts, networks, addresses, transactions, and deployment history.
- Monitor selected deployed-contract events and relevant transaction activity and generate configurable alerts and notifications.
- Maintain audit records and dashboards for lifecycle and operational visibility.
- Provide a restricted, read-oriented ChainOps AI Assistant for authorized operational queries.
- Evaluate the completed solution and prepare final project documentation.

5. Proposed Solution - ChainOps

The defining idea of ChainOps is end-to-end lifecycle traceability:



A user authenticates with ChainOps and works within an authorized organization and project. A developer connects an EVM-compatible smart-contract repository hosted on GitHub. ChainOps stores repository and project metadata and receives configured source-change information through GitHub webhooks. GitHub remains the source of truth for the source code.

When a relevant change occurs, ChainOps validates the event, identifies the exact commit revision, creates a pipeline run, and places long-running work in a background job. A worker obtains the exact repository revision in a temporary isolated execution environment. Existing smart-contract tooling such as Hardhat or Foundry compiles the Solidity contracts and runs automated tests. ChainOps captures statuses, logs, results, and relevant artifacts.

An established analyzer such as Slither performs static security analysis. ChainOps converts relevant findings into structured records. Its policy layer evaluates build, test, and security results against configured release rules. A revision may be allowed to continue, blocked, or sent for human review.

When the revision satisfies the required pipeline and policy conditions, an authorized user can create a deployment request. Where required, an authorized reviewer examines the validated revision, security findings, policy result, target network, and deployment information and approves or rejects the request.

For an approved deployment, ChainOps submits the deployment transaction to a configured EVM network through an RPC connection and tracks the transaction hash, pending state, confirmation, success, or failure. After confirmation, the Contract Registry records the deployed address, network, source revision, deployment transaction, ABI/artifact metadata, and deployment history.

The monitoring component observes selected contract events and relevant transaction activity. Events are decoded using contract ABI information and stored as structured runtime records. Configured rules may generate alerts, while in-app and/or email notifications inform authorized users about relevant pipeline, approval, security, deployment, and monitoring events. Important actions are recorded in an audit trail.

The ChainOps AI Assistant is restricted to read-oriented operational assistance using authorized ChainOps information. It may explain pipeline failures, security findings, policy blocks, deployment status, contract versions, and recent alerts, but it will not autonomously approve deployments or perform privileged blockchain transactions.

5.1 Main Functional Areas

Table 1: Main Functional Areas

Feature	What it Does
Authentication, Organization & Project Management	Authentication, organizations, projects, environments, roles, and permissions.
Repository Integration	Connect a GitHub smart-contract repository and receive relevant source-change information through webhooks.
Pipeline & Execution	Create and track asynchronous pipeline runs for exact source revisions.
Build & Testing	Use established tooling to compile Solidity contracts and run automated tests while collecting results and logs.
Security Validation	Run an established analyzer and represent findings as structured ChainOps records.
Policy & Release Gates	Evaluate validation/security results against project rules to allow, block, or require review.
Approval & Governance	Create deployment requests and allow authorized reviewers to approve or reject sensitive operations.
Deployment & Transaction Management	Submit eligible deployments to configured EVM networks and track transaction lifecycle.
Contract Registry	Maintain deployed addresses, networks, source revisions, artifacts/ABI metadata, transactions, and deployment history.
Blockchain Monitoring	Observe selected events and relevant transaction activity for registered contracts.
Rules, Alerts & Notifications	Generate alerts and in-app/email notifications from configured operational conditions.
Audit Trail & Reporting	Record important actions and provide lifecycle and operational summaries.
ChainOps AI Assistant	Provide restricted, read-oriented Q&A over authorized ChainOps operational data.

5.2 Project Scope

In scope: EVM-compatible Solidity projects; local EVM and selected public testnet deployment; authentication and role-based access; organization/project/environment management; GitHub integration; webhook/manual pipelines; exact source-revision tracking; asynchronous execution; temporary isolated workspaces; compilation/testing; static security-analyzer integration; structured findings; policy gates; approval workflow; EVM deployment and transaction tracking; Contract Registry; selected runtime monitoring; alert rules; notifications; audit history; dashboards; restricted AI assistant; and a controlled upgrade workflow as an advanced feature if schedule permits.

Out of scope: creating a blockchain, consensus protocol, Solidity compiler, or vulnerability analyzer; replacing GitHub or mature smart-contract tools; production-grade cross-chain bridges; support for every blockchain; handling real user funds; required mainnet deployment; autonomous AI approvals or privileged blockchain transactions; and building a general-purpose CI/CD platform for non-smart-contract projects.

5.3 Technology Boundary and Data Responsibilities

GitHub stores the source code, branches, commits, and repository history. ChainOps PostgreSQL stores platform data such as users, organizations, projects, integration metadata, pipeline runs, stage results, findings, policies, approvals, deployments, transactions, contract-registry records, monitoring configuration, selected runtime records, alerts, notifications, and audit records. Temporary execution storage is used only while checking out and processing an exact source revision. Selected long-lived ABI, bytecode, and build metadata may be stored using an artifact-storage mechanism finalized during architecture design. The blockchain network stores deployed contracts, on-chain transactions, contract state, and events. ChainOps does not permanently store entire GitHub repositories in PostgreSQL.

6. Resource Requirements

Table 2: Resource Requirements

Area	Suggested Technology
Frontend	React / Next.js + TypeScript
Backend API	Node.js + TypeScript; NestJS or equivalent
Platform Database	PostgreSQL
Queue / Background Jobs	Redis + BullMQ or equivalent
Smart Contracts	Solidity
Smart-Contract Tooling	Hardhat or Foundry
EVM Interaction	viem or ethers.js
Security Analysis	Slither or another established analyzer
Upgrade Tooling	OpenZeppelin Contracts / upgrade tooling where required
Source Integration	Git + GitHub API / Webhooks
Real-time Communication	WebSockets / Socket.IO where required
Notifications	In-app notifications + SMTP/email provider
Authentication / Access	Session/JWT-based authentication + RBAC
Containerization	Docker / Docker Compose
Blockchain Environment	Local EVM node + selected public EVM testnet(s)
AI Assistant	LLM API with restricted read-only access to authorized ChainOps data
Project Management / DevOps	GitHub + GitHub Actions
UI/UX	Figma

Hardware/connectivity requirements include development computers capable of running the selected stack and Docker, reliable Internet connectivity, and a suitable development/test environment.

7. Work Distribution

The project is divided into five primary technical ownership areas. Ownership does not mean isolated work; requirements, UML/design work, integration, reviews, testing, and overall system understanding remain shared team responsibilities.

Table 3 : Work Distribution

Member	Primary Ownership	Main Responsibilities
Member 1	Pipeline & Execution	Repository/webhook integration, pipeline lifecycle, background jobs/workers, source-revision checkout, build/test flow, pipeline history and status.
Member 2	Blockchain Deployment & Contract Registry	Deployment flow, EVM interaction, transaction/confirmation tracking, deployment failures, Contract Registry, versions, addresses and deployment history.
Member 3	Monitoring, Events & Alerts	Contract monitoring, event/log retrieval and decoding, runtime history, monitoring dashboard, alert rules and runtime status.
Member 4	Security & Governance	Security-analyzer integration, findings, release-policy evaluation, deployment gates, approval/rejection workflow, governance audit records, optional controlled upgrades.
Member 5	Platform, Access & AI Assistant	Authentication, organization/project/environment management, RBAC, shared platform integration, notifications, AI Assistant and restricted assistant access.

8.Reference

[1] Zou, W., Lo, D., Kochhar, P. S., Le, X.-B. D., Xia, X., Feng, Y., Chen, Z., & Xu, B. (2021). Smart Contract Development: Challenges and Opportunities. *IEEE Transactions on Software Engineering*. <https://doi.org/10.1109/TSE.2019.2942301>

9. Appendix

Table 4 : Project Timeline

Weeks	Phase	Activities and deliverables
1–4	Discovery and requirements	Study the domain; identify actors and use cases; prepare the SRS.
4–7	System design	Develop UML models, architecture, database design and UI/UX prototype.
8–11	Platform foundation	Implement authentication, organizations, projects, environments and role-based access.
10–13	Repository and pipeline	Connect GitHub; implement webhooks, background workers, exact-revision checkout, compilation and testing.
12–15	Security and governance	Integrate static analysis, structured findings, policy gates and deployment approval workflows.
14–17	Deployment and registry	Implement local/testnet deployment, transaction tracking and the versioned Contract Registry.
18–21	Monitoring and notifications	Capture and decode contract events; add monitoring dashboards, alert rules and notifications.
20–23	AI assistant and reporting	Add restricted read-only operational queries, audit history and lifecycle reporting.
24–27	System integration	Connect all modules and validate the commit-to-monitoring workflow.
26–29	Testing and refinement	Run functional, security, integration and usability tests; resolve defects.
29–32	Deployment and evaluation	Deploy the demonstration environment and evaluate against project objectives.
31–34	Final submission	Complete documentation, prepare the demonstration and retain time for final fixes.